DevOps roadmap: full-stack dev → production microservices on AWS

Starting point: 3+ years full-stack, comfortable with Jenkins and Docker basics, some AWS exposure. **Target:** Design and run a production-grade, distributed microservices platform with full CI/CD, IaC, observability, and security on AWS EKS.

Because you already know Docker and Jenkins fundamentals, this skips "what is a container" and goes straight into the depth you need for production systems. Each phase has: what to learn, why it matters at this level, and a hands-on project to prove it. Do the projects — reading about Kubernetes and running a real cluster are different skills.

Phase 0 — Fill the gaps (2–4 days)

Quick self-check before diving in. You likely have most of this already; skim and move on.

- **Linux**: process management, systemd, file permissions, networking tools (`netstat`, `ss`, `curl`, `dig`)
- **Networking**: DNS resolution, TCP vs UDP, TLS/SSL handshake, load balancing (L4 vs L7), what a reverse proxy does
- **Git**: rebasing, cherry-picking, trunk-based development vs GitFlow — you'll need a real branching strategy once multiple services and environments are involved

No project here — just make sure nothing above is a blind spot.

Phase 1 — Docker, beyond the basics (1 week)

You know `docker build` and `docker run`. Production usage needs more:

- Multi-stage builds (shrink a Node/Java/Python image from 900MB to 80MB)
- Layer caching strategy — order `COPY`/`RUN` so rebuilds are fast
- `docker-compose` for local multi-service dev environments
- Image tagging strategy (semver, git SHA, `latest` is not a deployment strategy)
- Pushing to and pulling from **Amazon ECR**
- Image vulnerability scanning with **Trivy**

Project: Take a 3-tier app (frontend + backend API + Postgres) you've built before. Containerize all three with multi-stage Dockerfiles, wire them together with `docker-compose`, and push the images to ECR with a proper tagging scheme.

Phase 2 — CI pipeline mastery (1–2 weeks)

You know basic Jenkins. Now build pipelines that are actually production-shaped:

- Jenkins **declarative pipelines** with stages: checkout → build → unit test → static analysis → image build → scan → push
- Jenkins shared libraries (reuse pipeline logic across microservices instead of copy-pasting Jenkinsfiles)
- Multi-branch pipelines (different behavior for `main`, `develop`, feature branches, PRs)
- Quality gates — fail the build on test failure, coverage drop, or critical vulnerability (SonarQube + Trivy in-pipeline)
- Get familiar with **GitHub Actions** too — many teams run it alongside or instead of Jenkins, and you'll want both in your toolkit

Project: A Jenkinsfile (as a shared library) that any of your microservices can reuse: lint → test → build image → scan → push to ECR, triggered automatically on every merge to `main`.

Phase 3 — Kubernetes fundamentals (2–3 weeks)

This is the biggest new topic. Go deep — everything downstream depends on it.

- Core objects: **Pod, Deployment, ReplicaSet, Service, Namespace**
- **ConfigMaps** and **Secrets** for externalized config
- **Ingress** and Ingress Controllers (specifically the **AWS Load Balancer Controller**, since you're on EKS)
- **Helm** — templating manifests, values files per environment, chart repos
- Resource requests/limits, liveness/readiness/startup probes
- `kubectl` fluency: `describe`, `logs`, `exec`, `port-forward`, `top`, debugging a `CrashLoopBackOff`

Project: Take last phase's containerized app, write Kubernetes manifests (then convert to a Helm chart), and deploy it to a local cluster (kind/minikube) before touching real AWS spend.

Phase 4 — EKS and AWS networking (2 weeks)

Now move to real infrastructure. This is where "Kubernetes" becomes "Kubernetes on AWS," and the two diverge in important ways.

- Creating an EKS cluster (via `eksctl` first, Terraform next phase)

- **VPC design for EKS**: public/private subnets across AZs, NAT gateways, security groups
- **IAM Roles for Service Accounts (IRSA)** — how pods get scoped AWS permissions without hardcoded keys
- **ECR** integration with EKS
- ALB Ingress Controller wiring an Application Load Balancer to your Ingress
- Node groups vs **Fargate** for EKS (when to use which)

Project: Stand up a real EKS cluster, deploy your Helm chart to it, and expose it via an ALB with a real domain (Route 53) and TLS (ACM).

Phase 5 — Infrastructure as Code with Terraform (2 weeks)

Everything above so far was done by hand or with `eksctl`. Now make it reproducible.

- Terraform core concepts: providers, resources, data sources, state
- **Remote state** with S3 + DynamoDB locking (critical for team use — never use local state for anything real)
- Writing reusable **modules** (one for VPC, one for EKS, one for RDS, etc.)
- Terraform workspaces or directory-per-environment for dev/staging/prod
- Importing existing resources vs starting clean

Project: Rebuild your entire Phase 4 infrastructure (VPC, EKS cluster, ECR, RDS) as Terraform modules. You should be able to `terraform destroy` and `terraform apply` and get back an identical environment.

Phase 6 — Microservices communication patterns (2 weeks)

This is the part that's specifically about *distributed* systems, not just "Kubernetes."

- Synchronous communication: REST and **gRPC** between services
- Asynchronous communication: **SQS/SNS** (AWS-native) or Kafka for event-driven flows
- API Gateway patterns (AWS API Gateway, or Kong/NGINX inside the cluster) as the single entry point
- **Service mesh** (Istio or Linkerd): mTLS between services, retries, timeouts, circuit breaking, traffic splitting — without changing app code
- Service discovery inside Kubernetes (DNS-based, and how the mesh changes this)

Project: Build 3 small microservices (e.g., `auth`, `orders`, `notifications`) where `orders` calls `auth` synchronously (REST/gRPC) and publishes events to SQS that

`notifications` consumes. Deploy all three to EKS with a service mesh handling mTLS between them.

Phase 7 — Observability (2 weeks)

You can't run distributed systems blind. This phase is non-negotiable for "production-grade."

- **Metrics**: Prometheus scraping your services + Grafana dashboards
- **Logs**: centralized logging — EFK/ELK stack, or CloudWatch Logs Insights if you want to stay AWS-native
- **Tracing**: distributed tracing across your 3 microservices with Jaeger or AWS X-Ray — this is what actually tells you *where* a slow request is stuck
- **Alerting**: Alertmanager or CloudWatch Alarms wired to Slack/PagerDuty

Project: Instrument your Phase 6 microservices with Prometheus metrics and distributed tracing. Build one Grafana dashboard and one alert (e.g., error rate > 5%) that actually fires.

Phase 8 — GitOps and progressive delivery (1–2 weeks)

Your CI pipeline currently pushes deployments. Production systems usually pull instead.

- **ArgoCD** or Flux: the cluster state is defined in Git, and the tool continuously reconciles the cluster to match
- Why pull-based GitOps is more secure and auditable than a Jenkins job with cluster credentials
- **Blue-green** and **canary deployments** — shipping to 5% of traffic before 100%
- Feature flags as a complement to (not replacement for) deployment strategy

Project: Replace your Jenkins `kubectl apply`/Helm-push step with ArgoCD watching your manifests repo. Implement a canary rollout for one service using Argo Rollouts or a service-mesh-based traffic split.

Phase 9 — Security / DevSecOps (2 weeks)

Everything before this works. This phase makes it safe to run with real user data.

- Image scanning gated **in the pipeline** (not just informational — actually fail the build)
- Secrets management: **AWS Secrets Manager** or HashiCorp Vault, never secrets in Git or plain Kubernetes Secrets at rest

- **Kubernetes RBAC** — least-privilege service accounts, not `cluster-admin` for everything
- **Network Policies** — which pods can talk to which, default-deny as a baseline
- Pod Security Standards (replacing the deprecated PodSecurityPolicy)
- Basic SAST (static analysis) and DAST (dynamic scanning) in the pipeline

Project: Audit and harden your whole setup: RBAC per service account, network policies restricting service-to-service traffic to only what Phase 6 actually needs, secrets moved to Secrets Manager, scanning gate that blocks merges on critical CVEs.

Phase 10 — Scalability and resilience (1–2 weeks)

The final layer: proving the system survives load and failure, not just that it runs.

- **Horizontal Pod Autoscaler (HPA)** and Vertical Pod Autoscaler
- **Cluster Autoscaler** or **Karpenter** for node-level scaling
- Load testing with k6 or Locust — know your system's actual breaking point
- **Chaos engineering:** Chaos Mesh or AWS Fault Injection Simulator — deliberately kill pods/nodes and watch the system recover
- Circuit breakers, retries with backoff, timeouts — revisit these from the service mesh angle now that you've seen real failure

Project: Load test your platform, tune HPA thresholds based on real numbers, then run a chaos experiment (kill a pod, kill a node) and confirm the system self-heals without manual intervention.

Capstone project

Put it all together: a small e-commerce-style platform (`auth`, `catalog`, `orders`, `payments`, `notifications`) with:

- Each service containerized, built and scanned via a shared Jenkins pipeline
- All infrastructure (VPC, EKS, RDS, IAM) defined in Terraform modules
- GitOps deployment via ArgoCD with canary rollouts
- Service mesh handling inter-service mTLS and traffic management
- Full observability: metrics, logs, and traces for every service
- Secrets in Secrets Manager, network policies enforced, RBAC scoped per service
- Autoscaling tuned from real load test results, verified to survive a chaos experiment

This is genuinely portfolio/interview-grade if you build it end to end — it demonstrates every skill a senior DevOps/platform engineer is expected to have.

Suggested timeline

Weeks	Focus
1	Phase 0 + Phase 1 (Docker deep dive)
2–3	Phase 2 (CI) + Phase 3 (K8s fundamentals)
4–5	Phase 4 (EKS) + Phase 5 (Terraform)
6–7	Phase 6 (microservices patterns) + Phase 7 (observability)
8–9	Phase 8 (GitOps) + Phase 9 (security)
10	Phase 10 (scalability/resilience)
11–12	Capstone project

~12 weeks at a steady pace alongside full-time work. Compress if you can dedicate more hours; the ordering matters more than the exact timing.

Core resources

- Kubernetes docs: kubernetes.io/docs
- AWS EKS docs: docs.aws.amazon.com/eks
- Terraform AWS provider docs: registry.terraform.io/providers/hashicorp/aws
- Istio docs: istio.io/latest/docs
- Prometheus docs: prometheus.io/docs
- ArgoCD docs: argo-cd.readthedocs.io
- "Kubernetes Up & Running" (O'Reilly) — solid book once you've done Phase 3 hands-on

Notes on sequencing

- Don't skip the local Kubernetes step (kind/minikube) in Phase 3 before touching EKS — debugging cluster networking issues is much cheaper on your laptop.
- Terraform (Phase 5) is placed after your first manual EKS cluster (Phase 4) on purpose — it's much easier to write IaC for infrastructure you've already built by hand once.
- Watch AWS spend from Phase 4 onward: EKS control plane + NAT gateways + ALBs are the recurring costs. Tear down (`terraform destroy`) between study sessions if you're not actively using the cluster.